

TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP.HCM
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO VỀ NGĂN XẾP (STACK)

Course Code: DASA230179

Giáo viên HD:	Th.S Vũ Đình Bảo	
Sinh viên:	Nguyễn Hùng Dũng	MSSV: 22134002
	Trần Như Hoàng	MSSV: 22134006
	Trần Nguyễn Phương Bình	MSSV: 24133006
	Phạm Ngọc Phúc	MSSV: 22134010
	Võ Hồng Quân	MSSV: 22134012

Academic Year 2025-2026

Mục lục

1	ĐỀ BÀI	3
2	BÀI LÀM	4
2.1	Thiết kế cấu trúc dữ liệu phù hợp để quản lý ghế trống	
2.1.1	Mảng hai chiều	4
2.1.2	Nested Dictionary	5
2.1.3	Chọn loại cấu trúc phù hợp	7
2.2	Thiết kế các tiêu chí chọn ghế cho người dùng	
2.2.1	Theo vị trí	7
2.2.2	Theo loại ghế	7
2.2.3	Theo trạng thái	8
2.2.4	Theo mức giá	8
2.2.5	Ví dụ vector biểu diễn	8
2.3	Viết thuật toán để tìm chỗ trống phù hợp	
2.3.1	Tự đánh giá giải pháp	10
2.4	Cách tư duy để tìm ra giải pháp	
2.5	Demo	
3	KẾT LUẬN	12
3.1	Kết quả đạt được	

ĐỀ BÀI

Bạn là một lập trình viên vừa mới nhận việc tại một rạp chiếu phim. Nhóm lập trình của bạn đang giải quyết bài toán **tìm chỗ phù hợp cho khách hàng**. Ứng dụng sẽ cung cấp một số tiêu chí để người dùng lựa chọn. Sau đó, ứng dụng sẽ **gợi ý danh sách các chỗ trống** phù hợp với các tiêu chí ấy.

Yêu cầu cụ thể:

1. Thiết kế cấu trúc dữ liệu phù hợp để quản lý ghế trống.
2. Thiết kế các tiêu chí chọn ghế để cho người dùng lựa chọn.
3. Viết thuật toán để tìm chỗ trống phù hợp với các tiêu chí đã chọn.
4. Tự đánh giá giải pháp của bạn (ưu điểm, nhược điểm, độ phức tạp, khả năng mở rộng...).
5. Trình bày cách bạn tư duy để tìm ra giải pháp này (quy trình phân tích, lựa chọn cấu trúc, so sánh phương án...).
6. Cài đặt một phiên bản demo (mã nguồn minh họa bằng ngôn ngữ lập trình, ví dụ: Python).

BÀI LÀM

2.1 Thiết kế cấu trúc dữ liệu phù hợp để quản lý ghế trống

Trong bài toán quản lý ghế trong rạp chiếu phim có 2 kiểu dữ liệu dễ quản lý nhất và dễ lưu trữ dữ liệu chính bao gồm: **mảng hai chiều (array 2D)** và **từ điển lồng nhau (nested dictionary)**.

2.1.1 Mảng hai chiều

Để giải quyết vấn đề chứa dữ liệu của ghế, mỗi ghế có thể được biểu diễn thành một **vector** chứa các thông tin liên quan được mã hoá như sau (các tiêu chí này là ví dụ minh họa nhanh, ở phần 2.2 sẽ thiết kế rõ ràng hơn), ví dụ minh họa trong hình 2.1:

- Cột 1: chỉ số hàng (A=1, B=2, ...)
- Cột 2: chỉ số cột (1, 2, 3, ...)
- Cột 3: loại ghế (0 = thường, 1 = VIP, 2 = Deluxe)
- Cột 4: trạng thái (0 = trống, 1 = đã đặt)
- Cột 5: giá tiền (phải theo từng loại ghế)

Khi gom tất cả vector lại, ta có một **mảng 2D**, trong đó: Mỗi hàng tương ứng với một ghế, mỗi cột là một thuộc tính (feature).

Ưu điểm:

- Truy cập nhanh theo chỉ số ($O(1)$).
- Có thể lưu ghế với số lượng không đồng đều ở từng hàng (không cần ma trận vuông).
- Thuận tiện cho xử lý bằng các thư viện ML/Data Analysis (NumPy, Pandas).

Nhược điểm:



Màn chiếu

A	14	13	12	11	10	9	8	7	6	5	4	3	2	1
B	14	13	12	11	10	9	8	7	6	5	4	3	2	1
C	14	13	12	11	10	9	8	7	6	5	4	3	2	1
D	14	13	12	11	10	9	8	7	6	5	4	3	2	1
E	14	13	12	11	10	9	8	7	6	5	4	3	2	1
F	14	13	12	11	10	9	8	7	6	5	4	3	2	1
G	14	13	12	11	10	9	8	7	6	5	4	3	2	1
H	14	13	12	11	10	9	8	7	6	5	4	3	2	1
J	14	13	12	11	10	9	8	7	6	5	4	3	2	1
K	14	13	12	11	10	9	8	7	6	5	4	3	2	1
L		12	11	10	9	8	7	6	5	4	3	2	1	

Ghế thường
 Ghế VIP
 Ghế Deluxe
 (Ghế Sweetbox cũng tại vị trí này)

Hình 2.1: Minh họa ghế bằng mảng hai chiều

- Ít linh hoạt khi cần thay đổi cấu trúc thông tin khi thêm thuộc tính mới như thêm ghế, không linh hoạt nếu số lượng ghế liên tục thay đổi.
- Nếu nhiều ghế trống hoặc không sử dụng, vẫn bị chiếm bộ nhớ vì các ghế phải xếp theo thứ tự trong hàng.

Ví dụ Python (dùng NumPy):

```

1 # [row, col, type, status, price]
2 seats = [
3     [1, 1, 0, 0, 100],    # A1 - thường, trống
4     [1, 2, 0, 1, 100],    # A2 - thường, đã dat
5     [5, 5, 1, 1, 150],    # E5 - VIP, đã dat
6     [12, 8, 2, 0, 200],   # L8 - Deluxe, trống
7 ]

```

2.1.2 Nested Dictionary

Tương tự như cách trên, khi giải quyết vấn đề chứa dữ liệu của ghế, mỗi ghế có thể được biểu diễn thành một **hashmap** (từ điển) chứa các thông tin liên quan, với key chính là mã ghế (ví dụ: A1, B5, L12). Ví dụ minh họa trong Hình 2.2.

Ưu điểm:

- Linh hoạt, dễ thêm bớt thông tin cho từng ghế.

- Truy cập trực tiếp bằng mã ghế ("A1", "B2", "VIP1").
- Thích hợp khi ghế không có dạng ma trận đều đặn (ví dụ ghế đôi, ghế VIP lẻ).
- Dễ lưu/đọc dữ liệu ở định dạng JSON.

Nhược điểm:

- Tốn bộ nhớ hơn so với array (do overhead của dictionary).
- Duyệt toàn bộ có thể chậm hơn khi cần tìm theo vị trí hàng-cột.



Hình 2.2: Minh hoạ bố trí ghế khi lưu trữ bằng Nested Dictionary

Ví dụ cấu trúc:

```

1 seats = {
2     "A1": {"row": 1, "col": 1, "type": "normal", "status": "free", "
3         price": 100},
4     "A2": {"row": 1, "col": 2, "type": "normal", "status": "booked",
5         "price": 100},
6     "E5": {"row": 5, "col": 5, "type": "VIP", "status": "booked", "
7         price": 150},
8     "L8": {"row": 12, "col": 8, "type": "Deluxe", "status": "free",
9         "price": 200}
10 }
```

2.1.3 Chọn loại cấu trúc phù hợp

Trong trường hợp cụ thể là **rạp chiếu phim**, số lượng và vị trí ghế luôn cố định, bố cục ghế được tổ chức rõ ràng theo hàng và cột. Vì vậy, lựa chọn **Array 2D** sẽ phù hợp hơn so với Nested Dictionary, do các lý do sau:

- Truy cập trực tiếp theo chỉ số hàng nên tốc độ $O(1)$.
- Cấu trúc ghế không thay đổi, không cần thêm bớt phần tử linh hoạt.
- Dễ dàng thao tác các phép xử lý đặc thù như tìm ghế liền kề, kiểm tra vùng trung tâm
- Dễ duyệt theo hàng/cột để tìm ghế trống.
- Tiết kiệm bộ nhớ hơn so với dùng nested dictionary.

2.2 Thiết kế các tiêu chí chọn ghế cho người dùng

Mỗi ghế trong rạp được biểu diễn thành một vector trong **array 2D**, trong đó mỗi cột (feature) tương ứng với một tiêu chí lựa chọn, bao gồm:

2.2.1 Theo vị trí

Khách hàng thường quan tâm đến trải nghiệm xem phim ở các vị trí khác nhau:

- 0 = gần màn hình: hình ảnh quá sáng, dễ gây mỏi mắt.
- 1 = trung tâm: vị trí tốt nhất, cân đối cả âm thanh và hình ảnh.
- 2 = xa màn hình: hình ảnh nhỏ hơn, có thể hơi mờ.
- 3 = ở rìa ngoài: góc nhìn bị lệch, dễ bị che khuất.

2.2.2 Theo loại ghế

Rạp chiếu phim thiết kế nhiều loại ghế để đáp ứng nhu cầu đa dạng:

- 0 = ghế thường: phổ thông, giá rẻ, phù hợp số đông.
- 1 = ghế VIP: vị trí đẹp, rộng rãi, dịch vụ tốt hơn.
- 2 = ghế đôi (Sweetbox): phục vụ cặp đôi, nhóm thân thiết.
- 3 = ghế Deluxe: cao cấp, tạo trải nghiệm riêng tư, thoải mái.

2.2.3 Theo trạng thái

Các trạng thái bao gồm:

- 0 = trống.
- 1 = đã đặt.

2.2.4 Theo mức giá

Mức giá vé cũng là một tiêu chí quan trọng:

- 0 = giá tối thiểu (rẻ nhất).
- 1 = giá trung bình (phổ biến).
- 2 = giá tối đa (đắt nhất).

2.2.5 Ví dụ vector biểu diễn

Giả sử mỗi ghế được biểu diễn thành một vector có 6 phần tử:

Ghế E5 (Hàng, Cột, Loại, Vị trí, Trạng thái, Giá) = [5, 5, 1, 1, 1, 150000]

Trong đó:

- Cột 0: số hàng ($E = 5$).
- Cột 1: số cột (5).
- Cột 2: loại ghế ($VIP = 1$).
- Cột 3: vị trí (trung tâm = 1).
- Cột 4: trạng thái (đã đặt = 1, trống = 0).
- Cột 5: giá vé (150000 VNĐ).

Kết luận: với cách mã hoá này, mỗi ghế trong rạp trở thành một hàng trong **array 2D**, dễ dàng cho việc lưu trữ, xử lý và tìm kiếm theo tiêu chí yêu cầu của khách hàng.

2.3 Viết thuật toán để tìm chỗ trống phù hợp

Mỗi ghế trong rạp được biểu diễn dưới dạng một vector:

$$[row, col, seat_type, position, status, price]$$

Trong đó:

- **row, col**: vị trí hàng và cột.
- **seat_type**: loại ghế (0 = thường, 1 = VIP, 2 = đôi, 3 = Deluxe).
- **position**: vị trí trong rạp (0 = gần màn hình, 1 = trung tâm, 2 = xa, 3 = rìa ngoài).
- **status**: trạng thái (0 = trống, 1 = đã đặt).
- **price**: giá vé.

Ví dụ dữ liệu ghế mẫu:

```
1 # each row: [row, col, seat_type, position, status, price]
2 seats = [
3     [5, 5, 1, 1, 0, 150000], # E5: VIP, center, available
4     [5, 6, 0, 1, 1, 100000], # E6: Normal, center, booked
5     [1, 1, 2, 3, 0, 200000], # A1: Sweetbox, edge, available
6     [3, 4, 0, 2, 0, 120000], # C4: Normal, far, available
7 ]
```

Thuật toán tìm tất cả các ghế phù hợp theo tiêu chí đầu vào được viết ngắn gọn bằng **list comprehension**:

```
1 def find_seats(data_seats, type_wanted, pos_wanted,
2               status_wanted, price_limit):
3     results = []
4     for seat in data_seats:
5         row, col, seat_type, position, status, price = seat #
6         # extract row, col, position, status, price
7         if (seat_type == type_wanted
8             and position == pos_wanted
9             and status == status_wanted
10            and price <= price_limit):
```

```

10         results.append(seat)
11     return results

1 results = find_seats(seats, type_wanted=1, pos_wanted=1,
    status_wanted=0, price_limit=150000)
2 print(results)
3 # results = [[5, 5, 1, 1, 0, 150000]]

```

2.3.1 Tự đánh giá giải pháp

Giả sử số lượng ghế trong rạp là n , thuật toán phải duyệt qua toàn bộ danh sách (brute force) ghế để kiểm tra điều kiện:

$$T(n) = O(n)$$

Trong đó:

- **Thời gian (Time Complexity):** $O(n)$ vì cần kiểm tra từng ghế một.
- **Không gian (Space Complexity):**
 - Nếu chỉ trả về ghế đầu tiên phù hợp: $O(1)$, do chỉ lưu một kết quả.
 - Nếu trả về tất cả ghế phù hợp (danh sách): $O(k)$, với k là số ghế thoả mãn điều kiện ($k \leq n$).

Nhận xét: Trong trường hợp quản lý các ghế ở rạp chiếu, độ phức tạp $O(n)$ là chấp nhận được, vì số lượng ghế trong một rạp chiếu phim thường nhỏ (khoảng vài trăm đến nghìn ghế).

2.4 Cách tư duy để tìm ra giải pháp

Để tìm ra giải pháp phù hợp, nhóm đã cân nhắc hai hướng tiếp cận chính:

- **Dùng Hash Map (dictionary):** Truy vấn nhanh theo **key**, ví dụ như tra cứu ghế trực tiếp qua mã ghế (E5, A1,...). Tuy nhiên, để xử lý các yêu cầu phức tạp hơn như lọc theo loại ghế, vị trí, trạng thái và giá tiền, vẫn cần duyệt qua toàn bộ dữ liệu. Do đó hiệu năng không khác biệt nhiều so với mảng khi áp dụng cho các điều kiện này.

-
- **Dùng Array/List (mảng 2D):** Là cách đơn giản nhất và trực quan nhất: mỗi ghế được biểu diễn bằng một danh sách `[row, col, seat_type, position, status, price]` và việc lọc dựa vào các điều kiện `if-else`. Độ phức tạp là $O(n)$ nhưng vẫn đảm bảo hiệu quả vì số lượng ghế trong rạp thường nhỏ (vài trăm đến một nghìn ghế).

2.5 Demo

Demo được thực hiện thông qua một notebook định dạng `.ipynb`, đi kèm là hai phiên bản tài liệu hỗ trợ: một bản dưới dạng Microsoft Word (`.docx`) và một bản notebook gốc (`.ipynb`) để dễ dàng tham khảo, chạy thử và chỉnh sửa.

KẾT LUẬN

3.1 Kết quả đạt được

Trong khuôn khổ bài tập, nhóm đã thiết kế và triển khai thành công một hệ thống quản lý ghế trong rạp chiếu phim với các thành phần chính như sau:

1. **Lựa chọn được cấu trúc dữ liệu phù hợp:** Sau khi so sánh giữa hai phương án: mảng hai chiều (`2D array`) và từ điển lồng nhau (`nested dictionary`) —> nhóm đã lựa chọn **mảng hai chiều** làm cấu trúc dữ liệu chính để lưu trữ thông tin ghế. Lý do là rạp chiếu phim có bố cục ghế cố định, không thay đổi thường xuyên, và yêu cầu truy xuất nhanh theo vị trí hàng–cột. Cấu trúc này giúp tối ưu hiệu năng, tiết kiệm bộ nhớ và hỗ trợ tốt cho các thao tác xử lý hàng loạt..
2. **Xây dựng mô hình biểu diễn ghế:** Mỗi ghế được mã hóa thành một vector gồm 6 thuộc tính: hàng, cột, loại ghế, vị trí trong rạp, trạng thái (trống/đã đặt) và giá vé. Việc chuẩn hóa thông tin dưới dạng số nguyên giúp đơn giản hóa quá trình so sánh và lọc dữ liệu, đồng thời đảm bảo tính nhất quán trong toàn bộ hệ thống.
3. **Triển khai thuật toán tìm kiếm ghế phù hợp:** Nhóm đã xây dựng một hàm tìm kiếm linh hoạt, cho phép người dùng lọc ghế dựa trên bốn tiêu chí: loại ghế, vị trí mong muốn, trạng thái (chỉ tìm ghế trống) và giới hạn giá. Thuật toán sử dụng phương pháp duyệt toàn bộ (*brute-force*) với độ phức tạp thời gian là $O(n)$, trong đó n là tổng số ghế. Với quy mô thực tế của một rạp chiếu phim (thường dưới 1.000 ghế), hiệu năng này là hoàn toàn chấp nhận được và đủ nhanh cho các ứng dụng thực tế.
4. **Triển khai và minh họa qua Jupyter Notebook:** Toàn bộ logic và ví dụ minh họa đã được triển khai đầy đủ trong file `.ipynb`. Điều này giúp dễ dàng kiểm chứng, chạy thử và mở rộng giải pháp.